§1 数据库系统概述

数据管理发展阶段：人工管理、文件系统、数据库系统

数据库系统组成：数据库、数据库管理系统、数据库用户

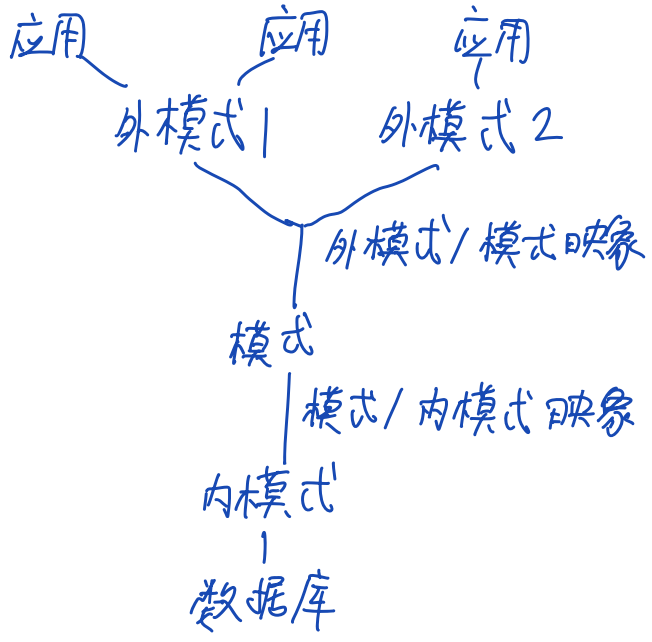
数据 —信息　　语法 —语义

数据模型：用来描述数据的语法、语义、操作

- 三要素：数据结构、数据操作、数据完整性约束
- 级别：概念数据模型、逻辑数据模型、物理数据模型
  - ↓ eg. E-R图

逻辑……：层次模型(树)、网状模型(图)、关系模型(表)

数据库系统三级模式：



外模式：视图层，应用相关数据逻辑表示，可以多个

内模式：物理层，数据物理结构和存储方式，只有一个

外模式/模式映象：模式变，外模式不变，保证逻辑独立性

模式/内模式映象：内模式变，模式不变，应用不变，保证物理独立性

DBMS：用户和OS之间，解释(灵活、效率低)、预编译

§2 关系模型

属性：列　域：列的取值　元组：行　关系：表

候选键：能确定一个元组的属性集　主键：候选键选一个

超键：包含候选键的属性集

模式：R(U, D, DOM, I, F)

U：属性　D：域　DOM：U→D　I：完整性约束　F：函数依赖集

关系类型：基本表、查询表、视图

关系完整性：

① 域完整性约束：取值在域内，NULL由语义决定

② 实体完整性约束：元组在主键取值不可重复、不可为空

③ 参考完整性约束：外键引用，可在不同/相同关系间，FK取值与CK相同或NULL

④ 用户定义的完整性约束：针对某一具体关系数据库的约束条件

关系模型：关系数据结构、关系操作集合、完系、完整性约束

关系代数：$\cup$、$-$、$\cap$、$\times$、σ、Π、$\bowtie$、$\div$

自然连接：消除重复属性的等值连接

外连接：保留不满足条件的元组，缺失取null，左、右、全 $R^*\bowtie S$

半连接：R.S自然连接结果在R上投影，$R \ltimes S$（不能交换）

元组关系演算：$\{t \mid P(t)\}$

§ 3

§4 数据依赖与关系模式规范化

问题：数据冗余、更新异常、插入异常、删除异常

函数依赖：$X \to Y$（X函数决定Y）

完全依赖：$X \xrightarrow{f} Y$　　部分依赖：$X \xrightarrow{P} Y$（类似CK与SK）

传递依赖：$X \to Y, Y \to Z, Y \not\to X$

Armstrong公理：①自反律：$Y \subseteq X$，则 $X \to Y$（Y是X的一部分）

②增广律：$X \to Y$，则 $XZ \to YZ$

③传递律：$X \to Y, Y \to Z$，则 $X \to Z$

④合并：$X \to Y, X \to Z$，则 $X \to YZ$

⑤伪传递：$X \to Y, WY \to Z$，则 $WX \to Z$

⑥分解：$X \to Y, Z \subseteq Y$，则 $X \to Z$

逻辑导出：$R\langle U, F\rangle$，$F \Rightarrow X \to Y$

逻辑蕴含：$F \models X \to Y$

属性集合的闭包 X_F^+：从X通过F能推出的所有属性

求 X_F^+：初始属性X，函数依赖集F

①取F中第1个决定子属性 $X^{(0)}$

②扫描F，找左边是 $X^{(0)}$ 一部分的（比如AB，找A→，B→，AB→）

③$X^{(i+1)}$ 为 $X^{(i)}$ U 能推出的属性

④如果 $X^{(i+1)} = X^{(i)}$ 或 $X^{(i)} = U$，则 $X^{(i)}$ 为 X_F^+，否则重来

最小函数依赖集（最小覆盖 F_{min}）：

①右部拆成单属性：$X \to ABL \Rightarrow X \to A, X \to B, X \to C$

②消除左部冗余：对不是单属性的左部 $X \to A$，去掉某个属性后

求闭包，若还包含A，则可删掉该属性

③消除冗余FD：对每个$X\to A$，令$G=F-\{X\to A\}$（去掉一个依赖），看A是否在X关于G的闭包里（$A\in X_G^+$?），如果是就可删该FD

是否无损分解：①关系R，n个属性，分成k个关系。关系在左，属性在上，建立k×n表格，如果属性是关系一部分，对应格填a_i（列），否则填b_{ij}（行列）

②用F中每个$X\to Y$，检查X对应的列，找出取值相等的行，有2个或以上相等，看对应行的Y列，如果有a就改a，全b就改相同b。

③如果某行变为$a_1\sim a_n$，则是无损分解。

只有2个关系模式是否无损分解：两关系的公共属性决定一个的独有属性

$(R1\cap R2)\to (R1-R2)/(R_2-R_1)$

保持函数依赖的分解：对每个分解出的小关系，计算其在函数依赖F上的投影，然后取并，如果逻辑蕴含F，则分解保持函数依赖F

求候选键：分四类：C_1：函数依赖中没有的属性

C_2：只在左出现　C_3：只在右出现　C_4：左右都出现

$(C_1UC_2)^+=U/C_4=\phi$，则C_1UC_2是候选键，

否则将C_4属性逐一加入C_1UC_2求闭包，重复该过程

1NF：每个关系的属性都是不可分的，比如日期→年月日

2NF：所有非主属性都完全依赖于候选键

3NF：非主属性既不部分依赖也不传递依赖候选键

BCNF：消除主属性对键的部分和传递依赖，左部必须是超键

判断范式：①求候选键

②确定主属性和非主属性

③1NF：看属性值域是否可分

④2NF：非主属性对键部分依赖

⑤3NF：非主属性对键传递依赖

⑥BCNF：主属性对键传递依赖

分解为3NF（无损，保持函数依赖）：

输入：$R\langle U,F\rangle$，输出：$\rho=\{R_0,R_1\dots R_n\}$

① 求F_{min}

② 不在F_{min}中出现的属性单独构成关系

③ 如果R中某个函数依赖包含U中全部属性则停止

④ 找左部相同的，划为一个关系

⑤保证有一个关系里存在候选键

分解BCNF：从左到右，遇到左边不是候选键就分解，$X\to A$

$\Rightarrow R_{i1}\{XA,F_{i1}\}$，$R_{i2}\{U_i-A,F_{i2}\}$，一直分直到都是BCNF.

§5 数据库设计

六阶段：需求分析 → 概念结构设计 → 逻辑结构设计 → 物理结构设计 → 数据库实施 → 数据库运行和维护

概念设计：E-R图

实体  属性  联系 

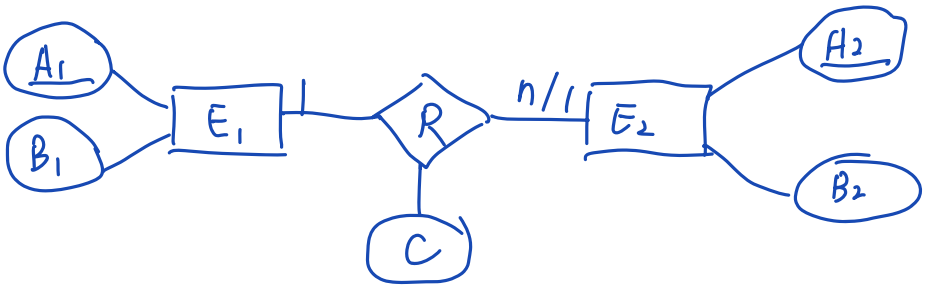
1:1  1:n  m:n 

设计方法：集中式模式设计法、视图集成法（局部视图设计、视图集成）

逻辑设计：将E-R图转为关系模式

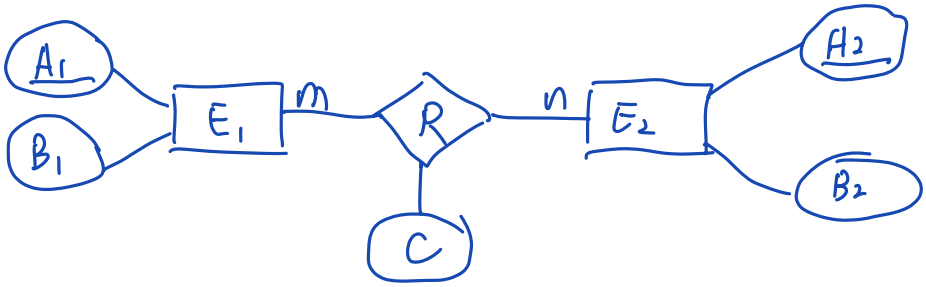
1:1

1:n



$E_1(\underline{A1}, B_1)$. $E_2(\underline{A_2}, B_2, \underline{A_1}, C)$

m:n



$E_1(\underline{A_1}, B_1)$ $E_2(\underline{A_2}, B_2)$

$R(\underline{A_1, A_2}, C)$

键完全相同，合并为新关系模式

物理设计：考虑存取时间、存储空间利用率、维护代价

存放位置：不同磁盘

存取方法：①索引：根据某属性建立索引

②聚簇：把某属性相同的元组在物理上放一起，一个关系一个聚簇

③hash

§7 查询处理与查询优化

查询处理的代价：总代价 = I/O代价 + CPU代价

多用户环境有内存代价，分布式环境有网络传输代价

代数优化：选择下推、投影下推、转为连接（语法树）

↳（σ和×一起）

物理优化：

(1) 选择：小关系 → 顺序扫描

无索引、无hash → 顺序扫描

满足条件元组比例较大 → 顺序扫描

主键等值查询 → 主键索引/hash

非主键等值查询，选择率低（<15%）→ 索引

非主键等值查询，选择率高（>15%）→ 聚簇索引，顺序扫描

范围查询 → 优先 B+树索引，hash不适用

AND → 有多属性索引优先用

聚集函数 → 只访问索引

(2) 连接：

①嵌套循环连接：顺序扫描外关系每个元组，与内关系中元组匹配

物理块少的为外关系

B_R：R的物理块数　B_S：S的物理块数　M：内存缓冲块数

若R为外关系：$B_R + \lceil \frac{B_R}{M-1} \rceil B_S$

若S为外关系：$B_S + \lceil \frac{B_S}{M-1} \rceil B_R$

⑤索引连接：顺序扫描外关系，对外关系的每个连接属性值，利用内关系连接属性上的索引或散列路径查找匹配元组

⑥排序归并连接：先按连接属性对两关系排序，然后按有序序列进行归并匹配

⑦散列连接：用同一个 hash 函数按连接属性把两个关系划分到桶中，然后只在相同桶内匹配元组

(3)选择连接算法：

①两表都已按连接属性排序：排序归并算法

②内表连接属性有索引或散列：索引连接/散列查找连接

③两表都小：嵌套循环连接

④大表等值连接且无索引无排序：散列连接

§8 事务管理

事务：数据库环境下由一组数据库操作序列组成的逻辑工作单元，是不可分割的整体。

符号：开始：BEGIN TRANSACTION

提交：COMMIT（正常结束）

回滚：ROLLBACK（异常，撤销事务）

事务特性ACID：原子性 Atomicity，一致性 Consistency，隔离性 Isolation，持续性 Durability

事务管理：恢复和并发控制

事务操作：read(x)：从数据库中将对象x读入read事务局部缓冲区

write(x)：从执行write事务的局部缓冲区中把对象x写入数据库

事务状态：→ 活动状态 → 局部提交状态 → 提交状态

↘ 失败状态 → 中止状态

故障：①事务故障：影响单个事务运行的故障（UNDO）

②系统故障：软硬件错误造成系统停止运转。

（UNDO未完成事务，REDO已完成事务）

③介质故障：外存爆了。（利用备份重构，REDO之后所有事务）

数据库备份：①脱机（静态）备份：不能被访问

②联机（动态）备份：可以被访问

③增量备份：备份上次更新的

④海量备份：备份全部

日志：事务故障、系统故障恢复，协助恢复介质故障

检查点：懒得写了

并发控制

并发导致的问题：① 丢失修改：

T_1	T_2
r(A)	
	r(A)
w(A)	
	w(A)

② 读脏数据：

T_1	T_2
r(A)	
w(A)	
	r(A)
ROLLBACK	

有ROLLBACK会出现

③ 不可重复读：

T_1	T_2
r(A)	
r(B)	
	r(B)
	w(B)
r(A)	
r(B)	

两次读结果不同

可串行化调度：调度结果与串行执行的结果相同，则调度正确

冲突可串行：冲突等价，调换调度中不冲突的

目标可串行：目标等价，调度结果相同

判断可串行：前趋图

把一个数据对象上的R、W操作全找出来，把冲突的（R-W，W-R，W-W）事务用箭头连起来，发生早的在前。如果最终图不存在回路，则可得到串行调度

锁：①X锁（排它锁）：写锁

②S锁（共享锁）：读锁

③U锁：写之前提前加，写入时变X锁

相容：

	S	U	X
S	√	√	×
U	√	×	×
X	×	×	×

加锁协议：①1级加锁协议：X锁，防止丢失修改

②2级加锁协议：S锁，读完释放，防止读脏数据

③3级加锁协议：S锁，事务结束后释放，防止不可重复读

2PLC（两段锁协议）：①扩展阶段：获得封锁，申请锁，不能释放

②收缩阶段：释放封锁，不能申请

↳ 可串行化，但可能死锁

死锁预防：一次封锁法、顺序封锁法、事务重执

$t_s(T_1) < t_s(T_2)$，T_1早，更年老，T_2年轻。T_B有锁，T_A申请

①等待-死亡策略（wait-die）：年老等待年轻

if $t_s(T_A) < t_s(T_B)$ then T_A waits;

else Rollback T_A, Restart T_A.

②伤-等待策略（wound-wait）：年轻等待年老

if $t_s(T_A) > t_s(T_B)$ then T_A waits

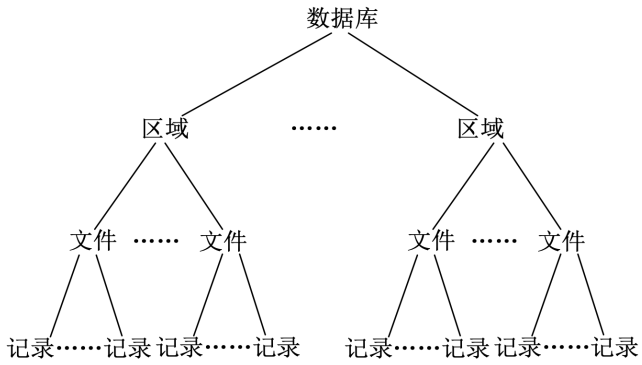
else Rollback T_B, Restart T_B.

多粒度封锁：

↓

显式封锁

隐式封锁



多粒度树的每个节点加锁，后裔必须加同样锁

意向锁：一个节点加意向锁，其下层节点将被加锁，反之亦然

① 意向共享锁IS锁：后裔加S锁

② 意向排它锁IX锁：后裔加X锁

③ 共享意象排它锁SIX锁：先加S锁，再加IX锁

相容性：

	S	X	IS	IX	SIX
S	√	×	√	×	×
X	×	×	×	×	×
IS	√	×	√	√	√
IX	×	×	√	√	×
SIX	×	×	√	×	×

基于时间标记的并发控制：

$t_s(T)$：事务T的时间标记，越小越老

$t_r(A)$：数据对象A的读时间标记，所有执行read(A)里最大的

$t_w(A)$：数据对象A的写时间标记，所有执行write(A)里最大的

读规则：$t_s(T) < t_w(A)$，则拒绝，否则读并更新 $t_r(A) = \max(t_s, t_r)$

写规则：$t_s(T) < t_r(A)$或$t_s(T) < t_w(A)$，则拒绝，否则更新 $t_w(A) = t_s(T)$